

Chapter 3

Will The Real Codeword Please Stand Still?: Stabilizer Codes

The treatment of QECCs in the last chapter was very general, but a bit unwieldy, particularly when it comes to finding new codes. We'd like a better method of discussing, manipulating, and finding QECCs, even if it comes at the cost of restricting somewhat the codes we can talk about. This chapter introduces the class of stabilizer codes, which have some nice properties and are much more tractable for most purposes than a general QECC. Stabilizer codes are built around a group of symmetries of the code that can distinguish between correct codewords and states with errors: The incorrect states will change under the action of the symmetry, while the correct ones will stay put.

3.1 The 9-Qubit Code Revisited

3.1.1 Error Syndrome Measurement for the 9-Qubit Code

The 9-qubit code is a stabilizer code, so I'll introduce the basic idea of a stabilizer code with some further examination of the nine-qubit code. Let's consider more carefully exactly what we measure when we measure the error syndrome for the nine qubit code. There are two parts to it: We measure each set of three qubits to see if one is different in the standard basis, and then we compare the three phases of the three sets of three qubits to see if one of the phases is different from the other two.

Recall that to determine if one of three qubits is different, we make two measurements. One measurement determines the parity of the first two qubits, and the other tells us the parity of the second and third qubits. Let us put that in a more quantum language: Determining the parity of two qubits is the same as measuring the eigenvalue of $Z \otimes Z$ on those qubits.

$$Z \otimes Z = \begin{pmatrix} +1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & +1 \end{pmatrix} \begin{array}{l} 00 \text{ (even)} \\ 01 \text{ (odd)} \\ 10 \text{ (odd)} \\ 11 \text{ (even)} \end{array} \quad (3.1)$$

Eigenvalue $+1$ corresponds to even parity (syndrome bit 0) and eigenvalue -1 corresponds to odd parity (syndrome bit 1). Thus, within each set of three, to figure out if one qubit is different in the standard basis, we should measure $Z \otimes Z \otimes I$ and $Z \otimes I \otimes Z$.

We also need to determine whether two sets of three have the same phase or opposite phase. We'd like to phrase this as measurement of an eigenvalue. It should probably involve X , since the eigenvalues of X tell us the phase in $|0\rangle \pm |1\rangle$, but we actually have an entangled state. To determine the phase of $|000\rangle \pm |111\rangle$ we should instead measure the eigenvalue of $X \otimes X \otimes X$. To determine whether two sets of three have the same phase or opposite phase, we should thus measure the eigenvalue of the tensor product of six X 's on

the nine-qubit code. The eight operators listed in table 3.1 are the *generators* of the stabilizer. Choosing a different way of defining the error syndrome of the code corresponds to choosing a different set of generators of the stabilizer.

Notice that we use tensor products of Z s to identify X (bit flip) errors, and we use tensor products of X s to identify Z (phase flip) errors. The pattern here is that the errors anticommute with the stabilizer elements used to find them, and it is this property that makes stabilizers useful for discussing quantum error-correcting codes.

3.2 Pauli Group

I warned that you would be sick of the Pauli operators by the end of this book, and this chapter will get you started. Stabilizer codes are built around the Pauli operators, as you can perhaps see from the example of the nine-qubit code. It's therefore worth examining them in more detail.

Definition 3.1. The *Pauli group* P_n is composed of tensor products of I , X , Y , and Z on n qubits, with an overall phase of ± 1 or $\pm i$.

I will refer to elements of the n -qubit Pauli group as “Pauli operators,” “Pauli errors,” or just “Paulis” in the future; if I need to make a distinction between P_1 and P_n , I will do so explicitly.

The Pauli group, as its name suggests, is a group: It is closed under multiplication since $Y = iXZ$, and similarly, the product of any two of X , Y , and Z is equal to the third with an overall factor of $\pm i$. In addition, $X^2 = Y^2 = Z^2 = I$. Any tensor product of Paulis is its own inverse, i.e., it squares to I , and if there is an overall phase factor of $\pm i$, it instead squares to $-I$. (I use the one-qubit I and the n -qubit identity I interchangeably.)

The one-qubit Paulis anticommute with each other

$$XZ = -ZX \quad (3.2)$$

$$YZ = -ZY \quad (3.3)$$

$$XY = -YX. \quad (3.4)$$

Of course, I commutes with X , Y , and Z , and any Pauli commutes with itself. More general pairs (P, Q) of operators from P_n always either commute ($PQ = QP$) or anticommute ($PQ = -QP$). The difference from the single qubit case is that for P_1 , only trivial pairs commute (when one Pauli is I or both are the same Pauli), but for $n > 1$, there are non-trivial commuting pairs. For instance, $X \otimes X$ commutes with $Z \otimes Z$. I will sometimes use the notation $[P, Q] = PQ - QP$ for the commutator and $\{P, Q\} = PQ + QP$ for the anticommutator.

P_n has 4^{n+1} elements since there are 4^n n -fold tensor products of I , X , Y , and Z , and 4 overall phases they could have. However, for many purposes, we can ignore the phase, giving us effectively 4^n distinct Paulis. If I wish to ignore the phase, I will refer to $\hat{P}_n$. Thus, $\hat{P}_n \cong P_n / \{I, iI, -I, -iI\}$. The elements of $\hat{P}_n$ are sets of 4 operators of the form $\{\pm P, \pm iP\}$, with P some n -qubit tensor product of I , X , Y , and Z . Each element of $\hat{P}_n$ can thus be associated with an unsigned Pauli operator P , and it is more convenient by far to refer to elements of $\hat{P}_n$ in terms of the associated Pauli rather than as a set of Paulis with signs.

Definition 3.2. Let $P \in P_n$. Then $\hat{P} \in \hat{P}_n$ is the element of $\hat{P}_n$ corresponding to P . Similarly, if S is a subset of P_n , then $\hat{S}$ is the subset of $\hat{P}_n$ consisting of $\hat{P}$ for all $P \in S$.

I have introduced these conventions separating P_n and $\hat{P}_n$ in order to be mathematically precise. In some cases, we need to work with elements or subsets of P_n , and in some cases, to get the details completely correct, it is better to work with $\hat{P}_n$. However, almost always, the distinction between the two is a small technical detail. My advice is to ignore the difference between P_n and $\hat{P}_n$ unless you get confused.

Any Pauli operator has eigenvalues $+1$ and -1 . Each eigenspace is exactly half the Hilbert space. That is, the $+1$ and -1 eigenspaces have dimension 2^{n-1} .

One feature of the Pauli group already mentioned is that P_n spans the space of all n -qubit linear operations. The weight t Paulis span the set of t -qubit errors. As discussed in corollary 2.5, this property means that to have a t -error correcting QECC, it suffices to correct all weight t Pauli operators.

This section has only discussed the most basic properties of the Pauli group. It has many more useful properties, some of which we will encounter in future sections of the book. Don't be surprised, however, if someday you are trying to solve a problem and discover a new property of the Paulis that is not mentioned in this book. In group theory, the Pauli group is an *extraspecial* group, and it truly deserves that name.

3.3 Stabilizer Codes

3.3.1 Definition and Basic Properties of the Stabilizer

Given a general quantum error-correcting code, we can define its stabilizer in more or less the same way as we did for the nine-qubit code:

Definition 3.3. Given a QECC $T \subseteq \mathcal{H}_{2^n}$, the *stabilizer* $S(T)$ is

$$S(T) = \{M \in P_n | M|\psi\rangle = |\psi\rangle \ \forall |\psi\rangle \in T\}. \quad (3.5)$$

In other words, the stabilizer is composed of the Pauli operators for which all codewords are $+1$ eigenstates. In principle, one could define a stabilizer consisting of all unitaries which fix every codeword, but it turns out to be most useful to concentrate on the Pauli operators.

Proposition 3.1. *The stabilizer $S(T)$ of a nonempty QECC T has three basic properties:*

- a) $-I \notin S(T)$
- b) $S(T)$ is a group
- c) $S(T)$ is Abelian

The property of being Abelian is the least obvious, but it makes sense since we want a set of operators which have the codewords as simultaneous eigenstates. Normally, for this to be possible, the operators should commute; in general, they only need to commute on a subspace, but for Pauli operators this is only possible if they truly commute.

Proof.

- a) $-I$ has no $+1$ eigenstates, so it cannot be in the stabilizer.
- b) If $M, N \in S(T)$, it is clear that their product is also in $S(T)$: For any $|\psi\rangle \in T$,

$$MN|\psi\rangle = M|\psi\rangle = |\psi\rangle. \quad (3.6)$$

- c) By the argument for property 2, $MN|\psi\rangle = NM|\psi\rangle$ for any $|\psi\rangle \in T$, $M, N \in S(T)$. Thus, $[M, N]$ annihilates the codewords. Paulis either commute or anticommute. If M and N anticommute, then $[M, N] = 2MN$, which is again an element of P_n , and therefore has no 0 eigenvalues. Therefore, the only option is that M and N commute.

□

It turns out to be most useful, when dealing with stabilizers, to work the other way around. We are given (or invent) a stabilizer with the properties we desire, and then use it deduce the code.

Definition 3.4. Let $S \subseteq P_n$ be an Abelian group, with $-I \notin S$. Then define the code space corresponding to S by

$$\mathcal{T}(S) = \{|\psi\rangle \text{ s.t. } M|\psi\rangle = |\psi\rangle \ \forall M \in S\} \quad (3.7)$$

X	Z	Z	X	I
I	X	Z	Z	X
X	I	X	Z	Z
Z	X	I	X	Z

Table 3.2: The generators for the five-qubit code.

In general, $T \subseteq \mathcal{T}(\mathcal{S}(T))$, but when they are actually equal, the code has special properties.

Definition 3.5. If T is a QECC with $T = \mathcal{T}(\mathcal{S}(T))$, it is a *stabilizer code*.

Stabilizer codes are also sometimes known as *symplectic codes*, *additive codes*, or $\text{GF}(4)$ *additive codes* for reasons that will be discussed in section 3.5 and section 5.2.

If we define a code from its stabilizer, then it is always a stabilizer code. In other words, a stabilizer code is one that could be defined just by giving its stabilizer. This is a consequence of the following proposition:

Proposition 3.2. $\mathcal{S} = \mathcal{S}(\mathcal{T}(\mathcal{S}))$.

I delay the proof until section 3.5 to keep you in suspense. Also, the proof will use a tool I won't introduce until then.

When dealing with stabilizer codes, I will frequently refer to the stabilizer $\mathcal{S}$ as the code instead of using the rather unwieldy phrase “stabilizer of the code $\mathcal{T}(\mathcal{S})$.” When $M \in \mathcal{S}$, I will call M a “stabilizer element” or some variant of that phrase. However, many people seem to find that terminology unwieldy too, and just call M a “stabilizer.” I do not approve of that usage, but I can't stop you if you want to say it.

Frequently we will want to pick a particular generating set $\{M_1, \dots, M_r\}$ for the stabilizer, as we did for the nine-qubit code. In a minimal generating set no generator is a product of other generators. When we take a product of generators, the order does not matter since the generators commute. Also, note that since $-I \notin \mathcal{S}$, all elements of $\mathcal{S}$ must square to I , and any power of a generator greater than 1 can be reduced. Therefore, the elements of the stabilizer are uniquely determined by taking a product

$$M_{i_1, \dots, i_r} = \prod_j M_j^{i_j}, \quad (3.8)$$

with $i_j \in \{0, 1\}$. This implies $|\mathcal{S}| = 2^r$.

Naturally, the set of generators is not unique, but when we do have a particular generating set to work with, I will talk about its elements as “stabilizer generators” or just “generators.” When you describe a stabilizer, almost always the easiest way to do so is by listing one particular set of generators. The generators are enough to define the whole stabilizer, and it is much easier to list r generators than to list all 2^r elements of the stabilizer.

The code space of a stabilizer code is defined by requiring that all the eigenvalues be +1, but there is nothing magical about the +1 eigenstates. After all, a -1 eigenstate of M is a +1 eigenstate of $-M$, which is still in $\mathcal{P}_n$. Given any set of generators $\{M_i\}$, we could define the code equally well by taking the -1 eigenstate, but switching M_i to $-M_i$. We *can't* arbitrarily change the eigenvalues associated with non-generators because the eigenvalues of the non-generators can be deduced from the eigenvalues of the generators. The generators are independent of each other, so we can pick their eigenvalues freely, but once we've done that, it defines the eigenvalues of all operators in the stabilizer. For more on this point, see section 3.5.

I'll conclude this subsection with a second example QECC to go with the nine-qubit code. This code has five physical qubits, and is, unsurprisingly, known as the “five-qubit code.” The stabilizer for the five-qubit code is given in table 3.2. The code is cyclic: if you permute the qubits cyclically, you'll get the same stabilizer. You can almost see this from table 3.2. Shifting by one qubit for generators one through three gives you generators two through four. However, the fifth permutation $Z \otimes Z \otimes X \otimes I \otimes X$ is not one of the generators. Nevertheless, it is still in the stabilizer, as the product of all four generators given in the table.

Even though it's not a generator, it still has the same status in the code as the four operators that are listed. Indeed, we could have chosen any four of these five operators as a set of generators without changing the code.

3.3.2 Projection Operator on a Stabilizer Code Subspace

In order to convert back from the representation of a stabilizer code in terms of its stabilizer to one as a subspace of a larger physical Hilbert space, we have definition 3.4, but it is sometimes helpful to have something more concrete. In particular, we'd like a projection operator that defines the subspace.

The codewords are supposed to be +1 eigenvectors of every element of the stabilizer, although it is sufficient to check that they are +1 eigenvectors of the stabilizer generators. The projector on the +1 eigenspace of generator M_i is $(I + M_i)/2$. Therefore, the projector on the code space of $\mathcal{S}$ is

$$\Pi_{\mathcal{S}} = \frac{1}{2^r} \prod_{i=1}^r (I + M_i), \quad (3.9)$$

when the stabilizer has r generators. Only states that are codewords — +1 eigenvectors of all generators — will avoid being annihilated by $\Pi_{\mathcal{S}}$, and any codeword will be left alone by $\Pi_{\mathcal{S}}$, as desired. Note that the order of the generators in the product does not matter since they all commute.

We can rewrite the projector $\Pi_{\mathcal{S}}$ in an interesting way by multiplying out the product. We get a sum of terms, each of which consists of a distinct product of the generators $\{M_i\}$. Based on equation (3.8), the products of the generators give us all elements of the stabilizer:

$$\Pi_{\mathcal{S}} = \frac{1}{2^r} \sum_{M \in \mathcal{S}} M. \quad (3.10)$$

We can come up with actual codewords for the code by applying $\Pi_{\mathcal{S}}$ to states in the physical Hilbert space and renormalizing. We might get 0, if the state we started with is orthogonal to the code space, but frequently we'll get a real codeword. For instance, for the five-qubit code, we can apply the projector to $|00000\rangle$ and $|11111\rangle$ to get two orthogonal codewords which can serve as a basis for the code space:

$$\begin{aligned} |\bar{0}\rangle &= |00000\rangle + |10010\rangle + |01001\rangle - |11011\rangle + |10100\rangle - |00110\rangle - |11101\rangle - |01111\rangle \\ &\quad + |01010\rangle - |11000\rangle - |00011\rangle - |10001\rangle - |11110\rangle - |01100\rangle - |10111\rangle + |00101\rangle \end{aligned} \quad (3.11)$$

$$\begin{aligned} |\bar{1}\rangle &= |11111\rangle + |01101\rangle + |10110\rangle - |00100\rangle + |01011\rangle - |11001\rangle - |00010\rangle - |10000\rangle \\ &\quad + |10101\rangle - |00111\rangle - |11100\rangle - |01110\rangle - |00001\rangle - |10011\rangle - |01000\rangle + |11010\rangle. \end{aligned} \quad (3.12)$$

I got these codewords by working my way through all 16 elements of the stabilizer for the five-qubit code (products of the generators given in table 3.2) and applying them to the starting states. You need to be careful of the signs involved, but otherwise the process is straightforward if tedious.

3.3.3 Distance and Size of a Stabilizer Code

When a stabilizer code is given either as a subspace or via the stabilizer, the number of physical qubits n is immediately obvious. The other two central properties (number k of logical qubits and distance d) are not so obvious, but can be deduced directly from the stabilizer.

Proposition 3.3. *If the stabilizer $\mathcal{S}$ on n physical qubits has $|\mathcal{S}| = 2^r$ (i.e., $\mathcal{S}$ has r generators), then $\mathcal{T}(\mathcal{S})$ has dimension 2^{n-r} . That is, there are $k = n - r$ logical qubits.*

Intuitively the result is easy to understand. The first generator of the stabilizer divides the Hilbert space into a +1 eigenspace and a -1 eigenspace of equal size. The codewords live in the +1 eigenspace. Each additional generator divides the subspace available for codewords in half again, so the total available dimension is 2^{n-r} . Non-generators do not divide the space since their eigenvalues can be derived by looking

at the eigenvalues of the generators. This argument is not a proof, since we would need to show that the additional generators divide not just the full Hilbert space in half, but also all the smaller subspaces defined by the earlier generators. The actual proof is straightforward, but less intuitive:

Proof. The dimension of $\mathcal{T}(\mathbf{S})$ is equal to the trace of $\Pi_{\mathbf{S}}$, the projection operator onto $\mathcal{T}(\mathbf{S})$. Now,

$$\text{tr } \Pi_{\mathbf{S}} = \frac{1}{2^r} \sum_{M \in \mathbf{S}} \text{tr } M. \quad (3.13)$$

However, $\text{tr } P = 0$ for all Paulis P except for the identity. Thus,

$$\text{tr } \Pi_{\mathbf{S}} = \frac{\text{tr } I}{2^r} = \frac{2^n}{2^r}. \quad (3.14)$$

□

You can check the nine-qubit code in this formula: 9 physical qubits, 8 stabilizer generators, and 1 encoded qubit. For the five-qubit code, we have four generators, so again there should be 1 encoded qubit, with the basis codewords we saw above.

There's one special case which is not, strictly speaking, a QECC, but is interesting nonetheless. Yes, it's true, there are some things which are interesting other than quantum error correction. When the stabilizer has n generators on n qubits, proposition 3.3 would tell us we have 0 encoded qubits, which is a Hilbert space of dimension 1. That is, it is a single state, up to normalization.

Definition 3.6. A *stabilizer state* is the code space of a stabilizer with n generators on n qubits.

Since $r = n$ is the maximum number of generators you can have, a stabilizer state is an extreme limit of a QECC. Some constructions of QECCs will alter the number of encoded qubits from another code, and sometimes a stabilizer state can be the starting or ending point of such a construction. Also, stabilizer states are fairly common in the theory of quantum information, even discounting states arising from quantum error correction. For instance, the GHZ state $|000\rangle + |111\rangle$ and the Bell states $|00\rangle \pm |11\rangle$, $|01\rangle \pm |10\rangle$ are stabilizer states.

To understand how to deduce the distance of a stabilizer code, let us go back to the nine-qubit code and recall how it handled errors. The generators of the stabilizer were used to give us bits of the error syndrome. In particular, some generators were able to signal the presence of certain errors while missing other errors, but by looking at the full set of generators, we were able to identify all of the single-qubit errors.

I mentioned that the property responsible for determining whether a generator M is useful for an error E is anticommutation. Let us see how this works. Suppose $M \in \mathbf{S}$ and $E \in \mathbf{P}_n$ anticommutes with M . Then for any $|\psi\rangle \in \mathcal{T}(\mathbf{S})$,

$$M(E|\psi\rangle) = -EM|\psi\rangle = -E|\psi\rangle. \quad (3.15)$$

$|\psi\rangle$ was a +1 eigenvector of M — that is the definition of the code space — but $E|\psi\rangle$ is a −1 eigenvector.

Conversely, if E commutes with M then

$$M(E|\psi\rangle) = EM|\psi\rangle = E|\psi\rangle. \quad (3.16)$$

One advantage to dealing with the Pauli group is that these are the only choices. Either M and E commute or they anticommute. If we have an error that commutes with M , M retains a +1 eigenvalue, whereas if the error anticommutes with M , M 's eigenvalue becomes −1, signaling that an error has occurred.

Definition 3.7. The *normalizer* $\mathbf{N}(\mathbf{S})$ of the stabilizer $\mathbf{S}$ is

$$\mathbf{N}(\mathbf{S}) = \{N \in \mathbf{P}_n | NM = MN \ \forall M \in \mathbf{S}\}. \quad (3.17)$$

If you know some group theory, you might recognize this as the definition of the *centralizer* of S (the set of things that commute with all elements of S) rather than the *normalizer* (the set of things that preserve S under conjugation), but because Paulis either commute or anticommute and $-I \notin S$, they are the same thing for a stabilizer. I am choosing to call it the normalizer rather than the centralizer because the normalizer relates to logical operations, and this is an important function of $N(S)$, as we shall see shortly in section 3.4.2.

Since the stabilizer is Abelian, $S \subseteq N(S)$ always. Indeed, $N(S)$ also contains $-S$ and $\pm iS$. Since we worry about eigenvectors of S , changing the sign of an operator in the stabilizer is important, but $N(S)$ is about errors, and global phase no longer matters. Therefore, we will usually want to work with $\hat{N}(S)$.

The normalizer tells us which errors can be detected by the stabilizer code. If a Pauli E is outside the normalizer $N(S)$, then $E|\psi\rangle$ has an eigenvalue -1 for some $M \in S$, and thus is detected by measuring the eigenvalues of the stabilizer elements. Note that this is true for *any* codeword $|\psi\rangle$. Also note that it is sufficient to measure the generators of the stabilizer: If N commutes with M_1 and M_2 , it also commutes with $M_1 M_2$. Thus, if N commutes with all generators of S , then $N \in N(S)$.

When $E \in N(S)$, then $E|\psi\rangle$ has eigenvalue $+1$ for all $M \in S$, and therefore $E|\psi\rangle \in \mathcal{T}(S)$ for codewords $|\psi\rangle$. You might think that that means it is an undetectable error, but there is actually another class of Paulis that is “detectable” by definition 2.7. If $E \in S$, then, while it’s true that $E|\psi\rangle \in \mathcal{T}(S)$ for any codeword $|\psi\rangle$, it’s also true that $E|\psi\rangle = |\psi\rangle$, so E is not actually an “error”: it acts like the identity on the code space, leaving codewords unchanged. Ignoring global phases, we can say the same if $\hat{E} \in \hat{S}$.

Putting this together, we get a characterization of the detectable errors for a stabilizer code.

Theorem 3.4. *The set of undetectable errors for a stabilizer code S is $\hat{N}(S) \setminus \hat{S}$. The distance of S is $\min\{\text{wt } E | \hat{E} \in \hat{N}(S) \setminus \hat{S}\}$.*

The slanty line is a “set minus” operation. That is, $\hat{N}(S) \setminus \hat{S}$ consists of those elements of $\hat{N}(S)$ that are not in $\hat{S}$.

Proof. We can prove the first statement directly from the definition of the set of detectable errors (definition 2.7). The second statement will then follow immediately from the definition of distance. We need to consider three cases for E . In cases 1 and 2, $|\psi\rangle$ and $|\phi\rangle$ are arbitrary codewords.

1. Case 1: $\hat{E} \in \hat{S}$. Then for some choice of phase, $E \in S$ and $E|\phi\rangle = |\phi\rangle$, so

$$\langle\psi|E|\phi\rangle = \langle\psi|\phi\rangle, \quad (3.18)$$

and $c(E) = 1$. $\hat{E}$ is detectable.

2. Case 2: $\hat{E} \notin \hat{N}(S)$. Then $\exists M \in S$ such that $\{M, E\} = 0$ (for any choice of phase of E), so $M(E|\phi) = -E|\phi\rangle$, as per equation (3.15). Then

$$\langle\psi|E|\phi\rangle = \langle\psi|MEM|\phi\rangle = -\langle\psi|E|\phi\rangle = 0, \quad (3.19)$$

since $M^2 = I$ for stabilizer elements. Thus $c(E) = 0$ and E is detectable.

3. Case 3: $\hat{E} \in \hat{N}(S) \setminus \hat{S}$. Since $E \notin S$, $\exists$ codeword $|\phi\rangle$ such that $E|\phi\rangle \neq |\phi\rangle$. Let $|\psi\rangle = E|\phi\rangle$. Since $E \in N(S)$, $|\psi\rangle$ is also a codeword. However,

$$\langle\psi|E|\phi\rangle = 1 = \frac{1}{\langle\psi|\phi\rangle} \langle\psi|\phi\rangle, \quad (3.20)$$

whereas

$$\langle\phi|E|\phi\rangle = \langle\phi|\psi\rangle = (\langle\phi|\psi\rangle)\langle\phi|\phi\rangle. \quad (3.21)$$

Comparing equations (3.20) and (3.21) to definition 2.7 tells us that $\hat{E}$ is not detectable unless $|\langle\phi|\psi\rangle| = 1$, meaning $E|\phi\rangle = e^{i\theta}|\phi\rangle$. Furthermore, by equation (3.21), $\hat{E}$ is undetectable unless θ is the same for all $|\phi\rangle$. But that is not possible, since that would imply $\hat{E} \in \hat{S}$.

X	X	X	X
Z	Z	Z	Z

Table 3.3: The stabilizer for the four-qubit code.

□

The key point here is that when $E \in \hat{N}(S) \setminus \hat{S}$, then E maps some codewords to *different* codewords. That's the essence of an undetectable error, because the code has no way of knowing if a codeword is the original encoding of the state or the result of the action of the error on a different codeword.

Moving now to correcting errors, we find

Theorem 3.5. *The stabilizer code S corrects a set of errors $\mathcal{E} \subseteq P_n$ iff $\hat{E}^\dagger \hat{F} \notin \hat{N}(S) \setminus \hat{S}$ for all $E, F \in \mathcal{E}$.*

The theorem follows immediately from theorem 3.4 by comparing definition 2.7 with theorem 2.7. Recall that by the linearity of QECCs, and particularly corollary 2.5, it suffices to consider Pauli errors to understand the error-correcting capabilities of the code, at least when we are interested in correcting t -qubit errors.

For stabilizer codes, we have a slightly different notation than the more general $((n, K, d))$ notation that applies to arbitrary QECCs. Since the encoded subspace has a dimension that's always a power of 2, we write the code in terms of the number of encoded qubits k rather than the dimension $K = 2^k$ of the logical Hilbert space. We also use square brackets to indicate that we are dealing with a stabilizer code.

Notation 3.8. A stabilizer code with n physical qubits, k logical qubits, and distance d is denoted as an $[[n, k, d]]$ code. If the distance is unknown or irrelevant, it is an $[[n, k]]$ code.

We know that the nine-qubit code corrects a single-qubit error, so it must have distance at least 3. In fact, there are some 3-qubit Paulis (such as $X_1 X_2 X_3$) in $\hat{N}(S) \setminus \hat{S}$ for the code, so the distance is exactly 3. Thus, the nine-qubit code is a $[[9, 1, 3]]$ code. The five-qubit code also turns out to have distance 3, so it is a $[[5, 1, 3]]$ code. Note that these codes are also $((9, 2, 3))$ and $((5, 2, 3))$ codes, but the more specific notation for a stabilizer code is generally used for them. As it happens, the five-qubit code is the smallest distance 3 code. You'll see the proof of that, as well as other techniques for proving limits on QECCs, in chapter 7.

You might wonder about distance 2 codes, since all the QECCs we have seen so far have distance 3. Distance 2 codes tend to be much simpler, but they can still be interesting. After all, a distance 2 code can detect one error or correct one erasure. The smallest distance 2 code is a $[[4, 2, 2]]$ code given in table 3.3. It is not hard to see that any one-qubit Pauli will anticommute with one or both of the two generators, but there are some two-qubit Paulis (e.g., $X \otimes X \otimes I \otimes I$) that commute with both. Thus, the code has distance 2.

3.3.4 Degeneracy and Stabilizer Codes

When is a stabilizer code degenerate? In the proof of theorem 3.4, we implicitly calculated the matrix C_{ab} . In particular, we found that $C_{ab} = 0$ if $E_1^\dagger F \notin N(S)$ and $C_{ab} = 1$ if $E_1^\dagger F \in S$. Any set of Paulis is linearly independent, so we just need to check if the resulting C_{ab} has maximum rank.

If $E_1^\dagger E_2 \in S$, then $E_1^\dagger F \in S \Leftrightarrow E_2^\dagger F \in S$, so in this case the rows (or columns) of C_{ab} associated with E_1 and E_2 are the same. When $E_1^\dagger E_2 \notin S$, then only one (or neither) of $E_1^\dagger F$ and $E_2^\dagger F$ can be in S , so in this case the rows associated with E_1 and E_2 cannot have 1s in the same column. Thus, we get the following proposition:

Proposition 3.6. *A stabilizer code S is degenerate for the error set $\mathcal{E}$ iff $\exists E_1, E_2 \in \mathcal{E}$ with $E_1^\dagger E_2 \in S$.*

As with general QECCs, we can define degeneracy as a property of the code subspace without referring to specific set of errors by considering the distance. In the case of stabilizer codes, motivated by the above analysis, we can also sensibly define degeneracy when the code is used for error detection rather than error correction.

Definition 3.9. A stabilizer code S with distance d is *degenerate* if $\exists M \in S$, $M \neq I$, with $\text{wt } M < d$.

Note that this is slightly more general than the generic notion of a degenerate code. A stabilizer code with distance $d = 2t + 2$ can be degenerate if S contains any elements of weight $2t + 1$, but degeneracy wasn't defined for general non-stabilizer QECCs of even distance.

Looking once more at the stabilizer of the nine-qubit code (table 3.1), it is immediately obvious that it is a degenerate code. There are many generators of weight 2 and the code has distance 3. However, it is not always obvious from looking at the generators whether a stabilizer code is degenerate or not. It may be that the set of generators given to you all have high weight, but there is some product of the generators that does not. The five-qubit code is non-degenerate because all of the operators in the stabilizer have weight 4, but to see that you need to do some work.

3.4 Cosets and Error Syndromes

3.4.1 The Error Syndrome and the Stabilizer

For the nine-qubit code, we figured out the stabilizer by thinking about what we wanted to measure for the error syndrome. Each generator corresponded to a bit of the error syndrome. In fact, this is completely general:

Definition 3.10. Suppose S is a stabilizer code with generators $M_1, \dots, M_r$ and $|\psi\rangle$ is an eigenvector (not necessarily with eigenvalue +1) of all $N \in S$. The *error syndrome* of $|\psi\rangle$ is an r -bit string with i th bit 0 if $|\psi\rangle$ is a +1 eigenvector of M_i and i th bit 1 if $|\psi\rangle$ is a -1 eigenvector of M_i . The error syndrome $\sigma(E) : P_n \rightarrow \mathbb{Z}_2^r$ is the error syndrome of $E|\phi\rangle$ for any codeword $|\phi\rangle$. When the stabilizer associated with a syndrome is in doubt, we will write $\sigma_S(E)$.

If $P, Q \in P_n$, then $c(P, Q) = 0$ if P and Q commute and $c(P, Q) = 1$ if they anticommute. ($c(P, Q) : P_n \times P_n \rightarrow \mathbb{Z}_2$.)

For a Pauli, bit i of the error syndrome is the same as $c(E, M_i)$. Note that the syndrome of E will be the same no matter what codeword $|\psi\rangle$ we choose to evaluate. This follows from equations (3.15) and (3.16) and means that Pauli errors map the code space, which is a subspace with +1 eigenvalue for all stabilizer elements, to a subspace that has a different set of eigenvalues, but for which each eigenvalue is still shared for all states in the subspace.

The subspaces derived this way are also error-correcting codes, but they are associated with different eigenvalues of the stabilizer. We can reinterpret them as traditional stabilizer codes (with all +1 eigenvalues) by replacing M_i with $-M_i$ whenever the i th syndrome bit is 1. Note that there are 2^r possible values of the error syndrome, and each subspace is isomorphic to the code space, which has dimension 2^k . When there are n physical qubits, $r = n - k$. Also, all the subspaces of different error syndromes are orthogonal to each other, since they have different eigenvalues. Thus, the full Hilbert space $\mathcal{H}_{2^n}$ decomposes as a direct sum of the 2^{n-k} subspaces associated with different syndromes.

Proposition 3.7. The $c(P, Q)$ and $\sigma(E)$ functions have the following properties. In all cases, $+$ refers to binary addition.

- a) $c(P_1 P_2, Q) = c(P_1, Q) + c(P_2, Q)$.
- b) $c(P, Q) = c(Q, P)$.
- c) $\sigma(EF) = \sigma(E) + \sigma(F)$.

These are straightforward to verify.

The various error syndromes are associated with different cosets of the normalizer in P_n .

Proposition 3.8. Let $E, F \in P_n$, and S be a stabilizer. Then F and E are in the same coset of $N(S)$ iff E and F have the same error syndrome.

Proof.

$\Rightarrow$: If E and F are in the same coset, then $F = EN$, with $N \in \mathbf{N}(\mathbf{S})$. Let $M \in \mathbf{S}$. Then $c(F, M) = c(E, M) + c(N, M)$. But $N \in \mathbf{N}(\mathbf{S})$, so $c(N, M) = 0$. Therefore the error syndromes of E and F are the same.

$\Leftarrow$: The argument works the other way too: Let $N = E^\dagger F$. If the error syndromes of E and F are the same, then $c(N, M) = 0$ for all $M \in \mathbf{S}$. Therefore, $N \in \mathbf{N}(\mathbf{S})$. □

In much the same way as the full Hilbert space can be written as a direct sum of subspaces associated to different syndromes, the whole Pauli group can be partitioned into cosets of $\mathbf{N}(\mathbf{S})$, each associated with a different syndrome. All the cosets are the same size and there are 2^r of them. As a consequence, we know the size of the normalizer:

Proposition 3.9. $|\mathbf{N}(\mathbf{S})| = 4 \cdot 2^{n+k}$ when $\mathbf{S}$ has n physical qubits and k logical qubits.

3.4.2 Cosets inside the Normalizer

We can similarly analyze the cosets of $\mathbf{S}$ in $\mathbf{N}(\mathbf{S})$.

Proposition 3.10. Let $N_1, N_2 \in \mathbf{N}(\mathbf{S})$. Then N_1 and N_2 are in the same coset of $\mathbf{S}$ iff $N_1|\psi\rangle = N_2|\psi\rangle$ for all codewords $|\psi\rangle$.

Proof. $N_1|\psi\rangle = N_2|\psi\rangle$ iff $|\psi\rangle$ is a $+1$ eigenstate of $M = N_1^\dagger N_2$. This is true for all codewords $|\psi\rangle$ iff $M \in \mathbf{S}$. However, N_1 and N_2 are in the same coset of $\mathbf{S}$ iff $N_2 = N_1 M$ with $M \in \mathbf{S}$. □

Recall that when $N \in \mathbf{N}(\mathbf{S})$, then $N|\psi\rangle$ is a codeword for any input codeword $|\psi\rangle$. Thus, N is a *logical operation*: it always maps codewords to (possibly different) codewords. Since the different representatives of a coset of $\mathbf{S}$ act the same way on codewords, they are all different realizations of the *same* logical operation. Thus, the set $\mathbf{N}(\mathbf{S})/\mathbf{S}$ is of particular interest, since it consists of the distinct logical operations performed by the normalizer. ($\mathbf{S}$ is a normal subgroup of $\mathbf{N}(\mathbf{S})$ by definition, so we can take this quotient without any difficulties.)

Theorem 3.11. Let $\mathbf{S}$ be a stabilizer with n physical qubits and k logical qubits. Then $\mathbf{N}(\mathbf{S})/\mathbf{S} \cong \mathbf{P}_k$.

The quotient group $\mathbf{N}(\mathbf{S})/\mathbf{S}$ can be taken to be the *logical Pauli group*, performing Paulis on the encoded qubits. The proof of theorem 3.11 will be in section 3.5.

We can also look at cosets of $\mathbf{S}$ in the full Pauli group $\mathbf{P}_n$, as in figure 3.2. Each coset of $\mathbf{N}(\mathbf{S})$ breaks up into cosets of $\mathbf{S}$, so we can associate each coset of $\mathbf{S}$ in $\mathbf{P}_n$ with an error syndrome, inherited from the coset of $\mathbf{N}(\mathbf{S})$ which it sits within. If we pick a particular representative E of the coset of $\mathbf{N}(\mathbf{S})$, then we can again identify the coset of $\mathbf{S}$ with a logical operation $\bar{P} \in \mathbf{N}(\mathbf{S})/\mathbf{S}$; the interpretation of the coset is then a combination of the logical operation $\bar{P}$ and the error E .

When we perform decoding on a stabilizer code, we do just this. For each error syndrome s , we assign a particular error E_s with $\sigma(E_s) = s$. If our syndrome measurement gives s , we assume the error actually was E_s and correct that. If the error was actually some E' with the same syndrome, we hope that E_s and E' are in the same coset of $\mathbf{S}$. If not, there has been a logical Pauli error, the one associated with the actual coset of $\mathbf{S}$ in which E' lies.

Theorem 3.12. If $\mathbf{S}$ is a non-degenerate stabilizer code, the error syndromes of all errors in the correctable error set $\mathcal{E}$ are distinct. If $\mathbf{S}$ is a degenerate code, E and F have the same error syndrome iff $\hat{E}^\dagger \hat{F} \in \hat{\mathbf{S}}$.

Proof. By proposition 3.8, two errors $E \neq F \in \mathcal{E}$ have the same error syndrome iff they are in the same coset of $\mathbf{N}(\mathbf{S})$, meaning $E^\dagger F \in \mathbf{N}(\mathbf{S})$. But both errors are correctable, and by theorem 3.5, $\hat{E}^\dagger \hat{F} \notin \hat{\mathbf{N}}(\mathbf{S}) \setminus \hat{\mathbf{S}}$, so $E^\dagger F \in \mathbf{N}(\mathbf{S})$ iff $\hat{E}^\dagger \hat{F} \in \hat{\mathbf{S}}$. Thus, the error syndromes of two correctable errors E and F are the same iff $\hat{E}^\dagger \hat{F} \in \hat{\mathbf{S}}$. If this ever occurs, the code is degenerate. □

syndrome 00 no correction	S	$\bar{X}$	FS	$F\bar{X}$	syndrome 10 correct as F
	$\bar{Z}$	$\bar{Y}$	$F\bar{Z}$	$F\bar{Y}$	
syndrome 01 correct as E	ES	$E\bar{X}$	GS	$G\bar{X}$	syndrome 11 correct as G
	$E\bar{Z}$	$E\bar{Y}$	$G\bar{Z}$	$G\bar{Y}$	

Figure 3.2: Cosets of the normalizer and stabilizer in the Pauli group. The errors E , F , and G are chosen as canonical errors for the syndromes 01, 10, and 11.

X	Z	Z	X	I
I	X	Z	Z	X
X	I	X	Z	Z
Z	X	I	X	Z
$\bar{X}$	X	X	X	X
$\bar{Z}$	Z	Z	Z	Z

Table 3.4: The generators for the five-qubit code supplemented by representatives for logical Paulis.

Applying this theorem is one way to see that the five-qubit code has distance 3 and is non-degenerate. There are 15 possible one-qubit errors (X , Y , Z on each of five qubits), plus the identity. There are 4 generators, so there are 16 error syndromes. If you list the syndromes of the 16 errors, you'll find that each one is unique. There are no syndromes left over, so the five-qubit code is known as a *perfect* code.

It is frequently helpful to also pick representatives of the cosets of $\mathbf{S}$ in the normalizer. These representatives don't have an interpretation in error correction, but they do give us concrete realizations of the logical Pauli group, which is helpful in fault-tolerant computation. For now, they are simply convenient computational tools. We don't actually need to pick representatives for every coset of $\mathbf{S}$. Since $\mathbf{N}(\mathbf{S})/\mathbf{S}$ has a group structure itself, it is sufficient to pick representatives of generators of $\mathbf{N}(\mathbf{S})/\mathbf{S}$ and let the generators of other cosets be determined by multiplication. In other words, we can pick representatives for $\bar{X}_i$ and $\bar{Z}_i$ for $i = 1, \dots, k$. (The subscript i here means the operator acts on the i th logical qubit, not the i th physical qubit.) I have done this for the five-qubit code and the four-qubit code in tables 3.4 and 3.5. Then, for instance, you can get $\bar{Y}$ for the five qubit code to be $i\bar{X}\bar{Z} = Y \otimes Y \otimes Y \otimes Y \otimes Y$.

You need to be careful when assigning cosets to elements of the logical Pauli group. Remember, $\mathbf{N}(\mathbf{S})/\mathbf{S} \cong \mathbf{P}_k$, and we'd like to realize this through a choice of representatives for the generating cosets. This means that the commutation and anticommutation relationships of the logical Paulis must be realized through the coset representatives. For instance, the coset representatives for $\bar{X}_i$ and $\bar{Z}_i$ must anticommute, while the coset representative of $\bar{X}_i$ must commute with the representatives for $\bar{X}_j$ or $\bar{Z}_j$ ($j \neq i$).

We could in principle do the same thing for $\mathbf{P}_n/\mathbf{N}(\mathbf{S})$, choosing representative errors only for the basis error syndromes and deducing the other errors by multiplication. However, in most cases, this is not a good thing to do. If our goal is to correct t errors for the maximum possible t , what we'd like to do instead is choose the lowest weight error in each coset of $\mathbf{N}(\mathbf{S})$ as its representative. If we rely on multiplication to tell us the coset representatives, we'll frequently get errors of higher weight than necessary, and then there will be some lower weight errors that won't get corrected properly.

	X	X	X	X
	Z	Z	Z	Z
$\overline{X}_1$	X	X	I	I
$\overline{X}_2$	X	I	X	I
$\overline{Z}_1$	I	Z	I	Z
$\overline{Z}_2$	I	I	Z	Z

Table 3.5: The stabilizer for the four-qubit code supplemented by representatives for logical Paulis.

3.4.3 Maximum Likelihood Decoding

I'd like to depart briefly from the convention of most of this chapter, for which we had some fixed set $\mathcal{E}$ of errors that we'd like to correct, and we won't settle for anything less than correcting all of them. For the purposes of this subsection, assume we instead have some n -qubit Pauli channel, with $P \in \mathcal{P}_n$ occurring with probability p_P .

Given a stabilizer code $\mathcal{S}$, we'd like to find a decoding procedure which minimizes the probability of error when the code goes through a Pauli channel. As discussed above, a decoding procedure can be understood as assigning to each error syndrome s a Pauli Q_s . When we measure s , we assume the actual error was Q_s and so perform the correction by inverting that error. If the actual error was something else, we may end up with the wrong state. In particular, if the true error P was in a different coset of $\mathcal{S}$ than Q_s was, we'll end up with some logical Pauli operation.

Thus, we can calculate the probability of logical error by summing over the cosets of $\mathcal{S}$ and $\mathcal{N}(\mathcal{S})$.

Proposition 3.13. *Let C_s be the coset of $\hat{\mathcal{N}}(\mathcal{S})$ with error syndrome s . $\hat{Q}_s\hat{\mathcal{S}}$ is the coset of $\hat{\mathcal{S}}$ containing the canonical error $\hat{Q}_s$ with syndrome s , so $\hat{Q}_s\hat{\mathcal{S}} \subseteq C_s$.*

a) *The probability of error syndrome s (regardless of whether we correctly identify the error) is*

$$p_s = \sum_{\hat{P} \in C_s} p_P. \quad (3.22)$$

b) *The probability of having syndrome s and no logical error (i.e., error correction is successful) is*

$$p_{s,\text{OK}} = \sum_{\hat{P} \in \hat{Q}_s\hat{\mathcal{S}}} p_P. \quad (3.23)$$

c) *The total probability of successfully decoding the state is*

$$p_{\text{OK}} = \sum_s \sum_{\hat{P} \in \hat{Q}_s\hat{\mathcal{S}}} p_P. \quad (3.24)$$

There is no interaction between the different error syndromes. If we change Q_s for one value of s , the only change to p_{OK} comes through the change to $p_{s,\text{OK}}$. We can therefore treat each syndrome separately to maximize $p_{s,\text{OK}}$. It doesn't matter which element of the coset $Q_s\mathcal{S}$ we choose, only which coset we choose within C_s .

In order to maximize the probability of successfully decoding, for each syndrome s , we should choose the coset $Q_s\mathcal{S}$ which maximizes $p_{s,\text{OK}}$. As given by equation (3.23), that just means we look at each coset and add up the probability of all Paulis in the coset. Then we choose the coset with the highest value to be the representative coset for s .

The maximum likelihood decoder should be contrasted with the decoder that optimizes the distance, for which we choose the coset containing the lowest-weight Pauli. If we have an independent Pauli channel, the

two procedures frequently, but not always, give the same answer: When the probability of error per qubit is small, a specific low-weight error will have higher probability than a specific high-weight error. However, a coset which contains one low-weight error and a bunch of high-weight errors may be less likely than a coset which contains a large number of medium-weight errors.

Getting the *exactly* optimal decoder, in the sense of always picking the most likely coset, is usually a computationally challenging task. Often, therefore, we are willing to settle for a good approximate decoder that gets p_{OK} to be close to the optimal value but with a much smaller computational cost.

3.5 Binary Symplectic Representation

3.5.1 The Symplectic Representation of Paulis

One of the most useful techniques when dealing with stabilizer codes is to ignore the phases of Paulis and work with $\hat{P}_n$ instead of P_n . We lose a couple of things by doing this. One is the ability to define the code space, which seems like a serious loss, but isn't really — as discussed above, the other eigenspaces of the stabilizer have the same error correction properties as the code space. The other missing thing is the ability to tell whether Paulis commute or anticommute, since $\hat{P}_n$ is an Abelian group. This is a much greater loss, so we'll need to find a substitute.

The structure of $\hat{P}_n$ is very straightforward. There are 4^n elements, and all pairs commute under multiplication. Therefore, $\hat{P}_n \cong \mathbb{Z}_2^{2n}$. The group operation for $\mathbb{Z}_2^{2n}$ is usually written as addition. Conventionally, we choose to represent elements of $\hat{P}_n$ as two n -bit binary vectors, one representing the “ X ” component and one representing the “ Z ” component.

Definition 3.11. The *binary symplectic representation* of $\hat{P}_n$ is an isomorphism between $\hat{P}_n$ and $\mathbb{Z}_2^n \times \mathbb{Z}_2^n$: $P \leftrightarrow v_P = (x_P | z_P)$. The i th component of x_P is 0 if P acts on qubit i as I or Z and 1 if P acts on qubit i as X or Y . The i th component of z_P is 0 if P acts on qubit i as I or X and 1 if P acts on qubit i as Y or Z . That is,

$$\begin{array}{ccc} I & & (0|0) \\ X & \longleftrightarrow & (1|0) \\ Y & & (1|1) \\ Z & & (0|1) \end{array} \quad (3.25)$$

For instance, $X \otimes I \otimes Y \leftrightarrow (1 \ 0 \ 1 | 0 \ 0 \ 1)$. The “symplectic” refers to the substitute for commutativity/anticommutativity:

Definition 3.12. The *symplectic form* (or symplectic product) on $\mathbb{Z}_2^n \times \mathbb{Z}_2^n$ is $(x_1 | z_1) \odot (x_2 | z_2) = x_1 \cdot z_2 + x_2 \cdot z_1$, where the dot product is the usual scalar product in the vector space $\mathbb{Z}_2^n$ and addition is modulo 2.

Proposition 3.14. $v_P \odot v_Q = c(P, Q)$ for $P, Q \in P_n$.

Proof. This can be checked exhaustively for a single qubit. For more than one qubit, note that both $\odot$ and $c(\cdot, \cdot)$ are equal to the parity of their single-qubit results. Therefore, since they are equal for single qubits, they are also equal for n qubits. $\square$

Combining the symplectic form with the binary symplectic representation recovers the primary structure of the Pauli group. We can switch back and forth between the two representations to use whichever one is the most convenient at the moment. The conversion process is summarized in table 3.6. The one thing that is lost in the transition is the phase of a Pauli, so you must be careful whenever dealing with something that depends on that.

Some of the conversions need a little more explanation. There were three conditions for S to be a stabilizer: that $-I \notin S$, that S is a group, and that S is Abelian. Since phase is lost when converting to the binary symplectic representation, the first condition is irrelevant. The second condition means that S becomes a linear subspace in $\mathbb{Z}_2^n \times \mathbb{Z}_2^n$, and the third means that $v \odot w = 0 \ \forall v, w \in S$. This analysis and the conversion of the normalizer prompts the following definition:

In the Pauli group	Binary symplectic representation
$P \in \mathbf{P}_n$	$v_P = (x_P z_P) \in \mathbb{Z}_2^n \times \mathbb{Z}_2^n$
Multiplication	Addition
$c(P, Q)$	$v_P \odot v_Q$
Phase	No equivalent
Stabilizer $\mathbf{S}$	Weakly self-dual subspace $\mathbf{S}$
Normalizer $\mathbf{N}(\mathbf{S})$	Dual subspace $\mathbf{S}^\perp$ (under $\odot$)
Minimal set of generators for $\mathbf{S}$	Basis for $\mathbf{S}$

Table 3.6: Equivalence between the Pauli group and its binary symplectic representation.

$$\begin{array}{c} \overline{X} \\ \overline{Z} \end{array} \left(\begin{array}{ccccc|ccccc} 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{array} \right)$$

Table 3.7: The binary symplectic representation of the stabilizer and logical Pauli operators for the five-qubit code.

Definition 3.13. Let V be a linear subspace of $\mathbb{Z}_2^n \times \mathbb{Z}_2^n$. The *dual* of V (with respect to $\odot$) is $V^\perp = \{w \in \mathbb{Z}_2^n \times \mathbb{Z}_2^n | w \odot v = 0 \ \forall v \in V\}$. A subspace is *self-dual* if $V^\perp = V$. A subspace is *weakly self-dual* if $V \subseteq V^\perp$.

Table 3.7 gives the five-qubit code in binary symplectic representation, including the logical $\overline{X}$ and $\overline{Z}$ operators.

Conversion can go the other way too. For instance, it is convenient to define a notion of a set of Paulis being independent based on the standard definition for binary vector spaces:

Definition 3.14. A set of Paulis $\{P_1, \dots, P_m\}$ is *independent* iff the vectors $\{v_{P_1}, \dots, v_{P_m}\}$ are linearly independent.

A set of Paulis is independent unless one of them is a product of others; this is equivalent to saying that a set of $2n$ -bit binary vectors is independent unless one of them is a sum of others. Since the binary vector space is $2n$ -dimensional, the maximum number of independent Paulis is $2n$, which is equal to the number of generators of the Pauli group.

3.5.2 Linear Algebra Lemma

The power of the binary symplectic representation is that we can use standard results from linear algebra to prove a number of properties of the Pauli group and stabilizers. In particular, we have the following lemma:

Lemma 3.15. Let $\{P_1, \dots, P_m\}$ be an independent set of Paulis on n qubits, and let s be an m -bit vector with components s_i . Then $\exists Q \in \mathbf{P}_n$ with $c(P_i, Q) = s_i$. In fact, there are 2^{2n-m} such Paulis.

Proof. Converting to the binary symplectic representation gives us m linearly independent vectors v_i , and we wish to find vector w such that $v_i \odot w = s_i$. Each of these conditions is a linear equation, and since the vectors v_i are independent, the system of linear equations is non-singular. There are m equations in a $2n$ -dimensional vector space, so the space of solutions has dimension $2n - m$. It is a binary vector space, so that corresponds to 2^{2n-m} solutions. $\square$

An important consequence of the lemma is that it tells us about the decomposition of the full physical Hilbert space into subspaces associated with the different error syndromes. I've already discussed this decomposition, but the missing piece is given by the following corollary of lemma 3.15:

Proposition 3.16. *Let S be a stabilizer with generators $\{M_i\}$. Then for any error syndrome s , $\exists P \in P_n$ with $\sigma(P) = s$.*

That is, not only is every Pauli outside $N(S)$ associated with an error syndrome, but every possible error syndrome is associated with some Pauli.

3.5.3 Consequences of the Lemma

Now it's time for some of the proofs I owe you. There is proposition 3.2, which claims $S = S(\mathcal{T}(S))$, and theorem 3.11, which says $N(S)/S \cong P_k$.

First, here is the proof that $N(S)/S \cong P_k$:

Proof of theorem 3.11. The most straightforward way to show this is by sequentially picking coset representatives for the cosets corresponding to $\bar{X}_i$ and $\bar{Z}_i$. The Pauli group P_k is determined, like any finitely-presented group, by its generators and relations between them. In the case of the Pauli group, the relations are

$$c(X_i, X_j) = 0 \quad (3.26)$$

$$c(Z_i, Z_j) = 0 \quad (3.27)$$

$$c(X_i, Z_j) = \delta_{ij}. \quad (3.28)$$

It is sufficient to consider only the generating cosets, since we can let the representatives of a product of cosets be the product of the representatives, as discussed in section 3.4.2. Provided we can choose $\bar{X}_i$ and $\bar{Z}_i$ with the correct commutation relations, that gives us an injective map of P_k into $N(S)/S$. We know that $|N(S)| = 4 \cdot 2^{n+k} = |S||P_k|$, so an injection has to be isomorphism.

Suppose we've chosen some independent set of $\bar{X}_i$'s and $\bar{Z}_i$'s with the correct commutation relations, and we wish to choose one more. The new logical Pauli must be in $N(S)$, so in particular, it must commute with all generators of the stabilizer. Second, it has a defined commutation relation with the already chosen logical Paulis. By lemma 3.15, there exists a Pauli that satisfies all of these constraints.

The only remaining thing to check is that the new logical Pauli can be chosen to be independent of the prior ones. To simplify the analysis, let us first pick all the $\bar{X}_i$'s, then the $\bar{Z}_i$'s. When we are picking $\bar{Z}_j$, it satisfies different commutation relations than all previously selected logical Paulis. In particular, $\bar{Z}_j$ anticommutes with $\bar{X}_j$, unlike all the previous logical Paulis and all the stabilizer generators. Thus, $\bar{Z}_j$ must be independent.

When we pick $\bar{X}_j$, this argument doesn't apply, since the new one will commute with all stabilizer generators and all previous logical Paulis. However, there are $n - k$ stabilizer generators and at most $k - 1$ logical Paulis already chosen, for a total of at most $n - 1$ constraints. By lemma 3.15, there are thus at least 2^{n+1} possible solutions. The group generated by the stabilizer and previous logical Paulis contains only 2^{n-1} operators, so there are possible choices for $\bar{X}_j$ that are independent of the previous choices. □

Finally, we can prove proposition 3.2, showing that $S = S(\mathcal{T}(S))$:

Proof of proposition 3.2. If $M \in S$, then certainly $M|\psi\rangle = |\psi\rangle$ for any $|\psi\rangle \in \mathcal{T}(S)$, so $S \subseteq S(\mathcal{T}(S))$. We need to show that if $N \notin S$, then $N \notin S(\mathcal{T}(S))$. If $N \notin N(S)$ then $N|\psi\rangle$ (for any codeword $|\psi\rangle$) has a different eigenvalue for some $M \in S$, and is therefore orthogonal to the code space, which in turn means $N \notin S(\mathcal{T}(S))$.

If S has n generators, that is all we need. Otherwise, if $r < n$, we still need to show that if $N \in N(S) \setminus S$, then $\exists |\psi\rangle \in \mathcal{T}(S)$ such that $N|\psi\rangle \neq |\psi\rangle$. By lemma 3.15, we can choose a $M \in N(S) \setminus S$ such that $\{M, N\} = 0$. Consider the modified stabilizer S' formed by adding M to S as a new generator. S' has $r + 1$ generators, so its code space has dimension 2^{k-1} (by proposition 3.3). Since $k \geq 1$, there is at least one state $|\psi\rangle$ (up to normalization) in the code space of S' . $|\psi\rangle$ is left fixed by all elements of S' , and in particular by the elements

of $S \subset S'$. Thus, $|\psi\rangle \in \mathcal{T}(S)$. However, N anticommutes with M , which is in the stabilizer of $|\psi\rangle$. Therefore, $N|\psi\rangle$ has eigenvalue -1 for M , and in particular is orthogonal to $|\psi\rangle$. It follows that $N \notin S(\mathcal{T}(S))$, which proves the proposition. □